Discussion

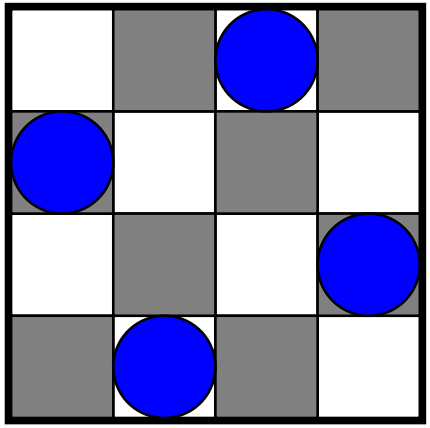
4 queens formalization

We have presented a Variable model: $X_i = j$, where i is the i-th column and i is the row

Constraints ($i, j = 1..4$ e $i < j$)

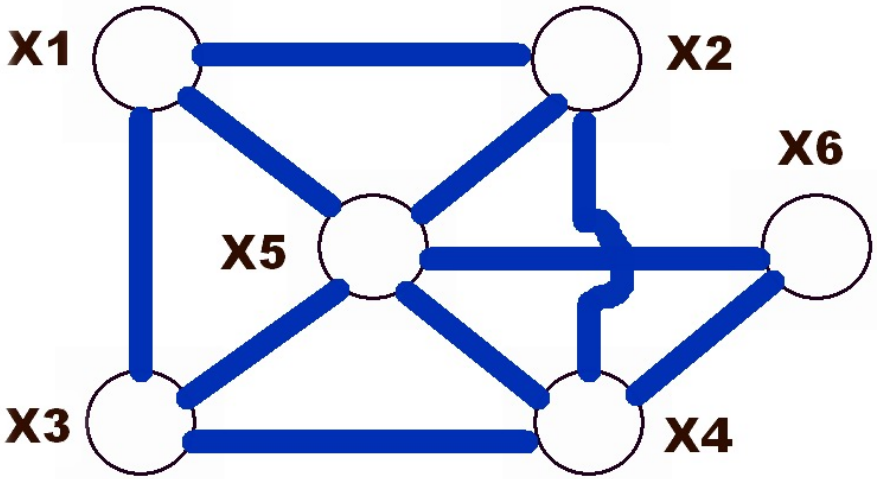
$$X_i \neq X_j \quad \text{(horizontal attack)}$$
$$|X_j - X_i| \neq j - i \quad \text{(diagonal attack)}$$

- What about:
- Variable model: $X_{i,j}$
- Domains: $D = 0, 1$
- Constraints?



NP-hardness of CSP

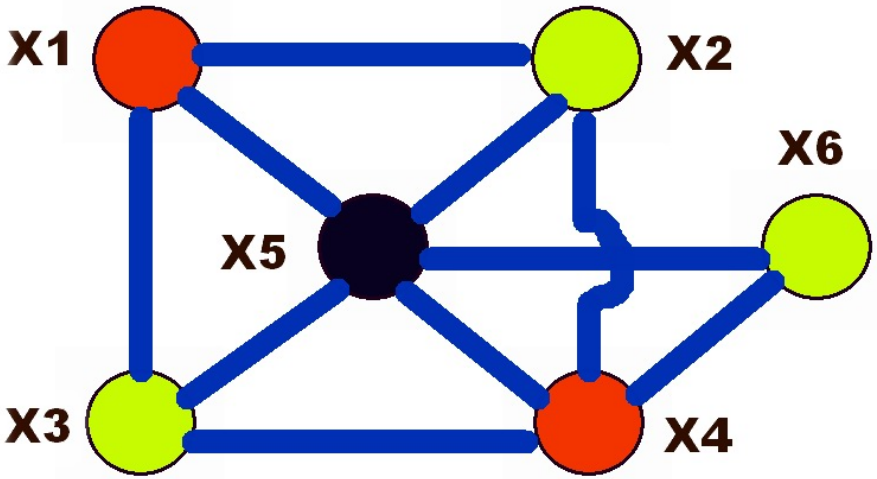
We can encode and solve a NP problem with a CSP.
3-coloring problem: does a coloring of the nodes of a graph, using three colors in such a way that adjacent nodes have different colors, exist?



$D_1 = \cdots = D_6 = \{\text{red}, \text{green}, \text{black}\}$,
$X_1 \neq X_2, X_1 \neq X_3, X_1 \neq X_5, \ldots, X_5 \neq X_6$

NP-hardness of CSP

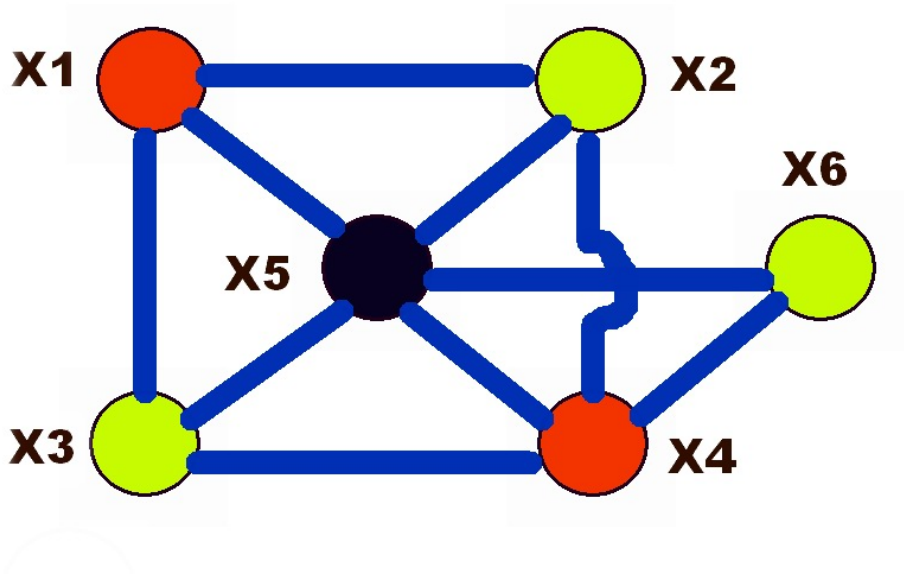
We can encode and solve a NP problem with a CSP.
3-coloring problem: does a coloring of the nodes of a graph, using three colors in such a way that adjacent nodes have different colors, exist?



$D_1 = \cdots = D_6 = \{\text{red}, \text{green}, \text{black}\}$,
$X_1 \neq X_2, X_1 \neq X_3, X_1 \neq X_5, \ldots, X_5 \neq X_6$

NP-hardness of CSP

We can encode and solve a NP problem with a CSP.
3-coloring problem: does a coloring of the nodes of a graph, using three colors in such a way that adjacent nodes have different colors, exist?



$D_1 = \cdots = D_6 = \{\text{red}, \text{green}, \text{black}\}$,
$X_1 \neq X_2, X_1 \neq X_3, X_1 \neq X_5, \ldots, X_5 \neq X_6$

A famous CSP

	1	2	3	4	5	6	7	8	9
1			6					9	
2				5		1	7		
3	2			9			3		
4		7			3			5	
5		2			9			6	
6		4			8			2	
7			1			3			4
8			5	2		7			
9		3					8		

$\text{domain}([X_{1,1}, \ldots, X_{9,9}], 1, 9)$
$X_{1,3} = 6, X_{1,8} = 9, X_{2,4} = 5, X_{2,6} = 1, X_{2,7} = 7, \ldots$
$\text{alldifferent}(X_{1,1}, \ldots, X_{1,9}) \ldots \text{alldifferent}(X_{9,1}, \ldots, X_{9,9})$
$\text{alldifferent}(X_{1,1}, \ldots, X_{9,1}) \ldots \text{alldifferent}(X_{1,9}, \ldots, X_{9,9})$
$\text{alldifferent}(X_{1,1}, \ldots, X_{3,3}) \ldots \text{alldifferent}(X_{7,7}, \ldots, X_{9,9})$

What's next

- We have seen what CSP/COP are and two naive approaches for solving them (generate/test and partial assignment/backtracking)
- We'll briefly see how to use a SAT solver
- Then, we'll have a quick view on Local Search and on Integer (Linear) programming (branch and bound)
- Then we'll start analyzing constraint programming (CP)

SAT Solving

SAT

Definition

- Given a set of Boolean variables $X_1, \ldots, X_n$ (i.e., $D_1 = \cdots = D_n = \{0, 1\}$)
- and a CNF (conjunctive normal form) propositional formula on these variables:

$$\Phi = (\ell_1^1 \vee \cdots \vee \ell_{n_1}^1) \wedge \cdots \wedge (\ell_1^k \vee \cdots \vee \ell_{n_k}^k)$$

  where each ℓ_j^i is a variable X_p or the negation of a variable $\neg X_p$
- The problem is that of establishing the existence of a satisfying assignment (for Φ) of the variables in $\{0,1\}$
- Let us observe that it is a CSP ($X_i \in \{0, 1\}$) and any clause is a constraint
- E.g. $(X_1 \vee \neg X_2)$ (equivalent to $\neg(\neg X_1 \wedge X_2)$) can be expressed, e.g., by $X_1 + (1 - X_2) \geq 1$ (i.e., $\{(1, 0), (1, 1), (0, 0)\}$.
- Remember that $0 \vee 0 = 0, 0 \vee 1 = 1 \vee 0 = 1 \vee 1 = 1$, $0 \wedge 0 = 0 \wedge 1 = 1 \wedge 0 = 0, 1 \wedge 1 = 1, \neg 0 = 1, \neg 1 = 0$

SAT

Definition

- Given a set of Boolean variables $X_1, \ldots, X_n$ (i.e., $D_1 = \cdots = D_n = \{0, 1\}$)
- and a CNF (conjunctive normal form) propositional formula on these variables:

$$\Phi = (\ell_1^1 \vee \cdots \vee \ell_{n_1}^1) \wedge \cdots \wedge (\ell_1^k \vee \cdots \vee \ell_{n_k}^k)$$

  where each ℓ_j^i is a variable X_p or the negation of a variable $\neg X_p$
- The problem is that of establishing the existence of a satisfying assignment (for Φ) of the variables in $\{0,1\}$
- Let us observe that it is a CSP ($X_i \in \{0, 1\}$) and any clause is a constraint
- E.g. $(X_1 \vee \neg X_2)$ (equivalent to $\neg(\neg X_1 \wedge X_2)$) can be expressed, e.g., by $X_1 + (1 - X_2) \geq 1$ (i.e., $\{(1, 0), (1, 1), (0, 0)\}$.
- Remember that $0 \vee 0 = 0, 0 \vee 1 = 1 \vee 0 = 1 \vee 1 = 1$, $0 \wedge 0 = 0 \wedge 1 = 1 \wedge 0 = 0, 1 \wedge 1 = 1, \neg 0 = 1, \neg 1 = 0$

SAT

- SAT was the first problem proved to be NP-complete (Cook)
- Thus, every instance of a problem in NP can be transformed in SAT instance
- A problem modeled as CSP (on finite domains) is typically in NP.
- Thus we can solve it by *reducing* (transforming) it to an instance of SAT
- Then we can use a SAT solver (e.g., MiniSAT, PicoSAT, RelSAT, etc.—see the SAT competition)
- We'll use clasp/clingo (we'll use it for ASP as well)
- Reductions can be not immediate ...

SAT

- SAT was the first problem proved to be NP-complete (Cook)
- Thus, every instance of a problem in NP can be transformed in SAT instance
- A problem modeled as CSP (on finite domains) is typically in NP.
- Thus we can solve it by *reducing* (transforming) it to an instance of SAT
- Then we can use a SAT solver (e.g., MiniSAT, PicoSAT, RelSAT, etc.—see the SAT competition)
- We'll use clasp/clingo (we'll use it for ASP as well)
- Reductions can be not immediate ...

SAT Reduction

4 Queens

- We should state that the variables X_1, X_2, X_3, X_4 have values in $\{1,\ldots,4\}$.
- For each of them (let us focus on the generic X_i) we introduce 4 Boolean Variables: $Z_1^i, Z_2^i, Z_3^i, Z_4^i$.
- Let us state that at least one of them is true: $Z_1^i \vee Z_2^i \vee Z_3^i \vee Z_4^i$.
- Let us state that two on the same column cannot be true: $\neg Z_1^i \vee \neg Z_2^i, \neg Z_1^i \vee \neg Z_3^i, \neg Z_1^i \vee \neg Z_4^i, \ldots, \neg Z_3^i \vee \neg Z_4^i$.
- Horizontal attack. For $i \neq j$: $\neg Z_1^i \vee \neg Z_1^j, \neg Z_2^i \vee \neg Z_2^j, \neg Z_3^i \vee \neg Z_3^j, \neg Z_4^i \vee \neg Z_4^j$.
- Diagonal attack. For $i \neq j$. Let $i = 2, j = 4$, then: $\neg Z_1^2 \vee \neg Z_3^4, \neg Z_2^2 \vee \neg Z_4^4, \neg Z_3^2 \vee \neg Z_1^4, \neg Z_4^2 \vee \neg Z_2^4$.

SAT Reduction

4 Queens

- We should state that the variables X_1, X_2, X_3, X_4 have values in $\{1,\ldots,4\}$.
- For each of them (let us focus on the generic X_i) we introduce 4 Boolean Variables: $Z_1^i, Z_2^i, Z_3^i, Z_4^i$.
- Let us state that at least one of them is true: $Z_1^i \vee Z_2^i \vee Z_3^i \vee Z_4^i$.
- Let us state that two on the same column cannot be true: $\neg Z_1^i \vee \neg Z_2^i, \neg Z_1^i \vee \neg Z_3^i, \neg Z_1^i \vee \neg Z_4^i, \ldots, \neg Z_3^i \vee \neg Z_4^i$.
- Horizontal attack. For $i \neq j$: $\neg Z_1^i \vee \neg Z_1^j, \neg Z_2^i \vee \neg Z_2^j, \neg Z_3^i \vee \neg Z_3^j, \neg Z_4^i \vee \neg Z_4^j$.
- Diagonal attack. For $i \neq j$. Let $i = 2, j = 4$, then: $\neg Z_1^2 \vee \neg Z_3^4, \neg Z_2^2 \vee \neg Z_4^4, \neg Z_3^2 \vee \neg Z_1^4, \neg Z_4^2 \vee \neg Z_2^4$.

SAT Reduction

4 Queens

- We should state that the variables X_1, X_2, X_3, X_4 have values in $\{1,\ldots,4\}$.
- For each of them (let us focus on the generic X_i) we introduce 4 Boolean Variables: $Z_1^i, Z_2^i, Z_3^i, Z_4^i$.
- Let us state that at least one of them is true: $Z_1^i \vee Z_2^i \vee Z_3^i \vee Z_4^i$.
- Let us state that two on the same column cannot be true: $\neg Z_1^i \vee \neg Z_2^i, \neg Z_1^i \vee \neg Z_3^i, \neg Z_1^i \vee \neg Z_4^i, \ldots, \neg Z_3^i \vee \neg Z_4^i$.
- Horizontal attack. For $i \neq j$: $\neg Z_1^i \vee \neg Z_1^j, \neg Z_2^i \vee \neg Z_2^j, \neg Z_3^i \vee \neg Z_3^j, \neg Z_4^i \vee \neg Z_4^j$.
- Diagonal attack. For $i \neq j$. Let $i = 2, j = 4$, then: $\neg Z_1^2 \vee \neg Z_3^4, \neg Z_2^2 \vee \neg Z_4^4, \neg Z_3^2 \vee \neg Z_1^4, \neg Z_4^2 \vee \neg Z_2^4$.

SAT Reduction

4 Queens

- We should state that the variables X_1, X_2, X_3, X_4 have values in $\{1,\ldots,4\}$.
- For each of them (let us focus on the generic X_i) we introduce 4 Boolean Variables: $Z_1^i, Z_2^i, Z_3^i, Z_4^i$.
- Let us state that at least one of them is true: $Z_1^i \vee Z_2^i \vee Z_3^i \vee Z_4^i$.
- Let us state that two on the same column cannot be true: $\neg Z_1^i \vee \neg Z_2^i, \neg Z_1^i \vee \neg Z_3^i, \neg Z_1^i \vee \neg Z_4^i, \ldots, \neg Z_3^i \vee \neg Z_4^i$.
- Horizontal attack. For $i \neq j$: $\neg Z_1^i \vee \neg Z_1^j, \neg Z_2^i \vee \neg Z_2^j, \neg Z_3^i \vee \neg Z_3^j, \neg Z_4^i \vee \neg Z_4^j$.
- Diagonal attack. For $i \neq j$. Let $i = 2, j = 4$, then: $\neg Z_1^2 \vee \neg Z_3^4, \neg Z_2^2 \vee \neg Z_4^4, \neg Z_3^2 \vee \neg Z_1^4, \neg Z_4^2 \vee \neg Z_2^4$.

SAT Reduction

4 Queens

- We should state that the variables X_1, X_2, X_3, X_4 have values in $\{1,\ldots,4\}$.
- For each of them (let us focus on the generic X_i) we introduce 4 Boolean Variables: $Z_1^i, Z_2^i, Z_3^i, Z_4^i$.
- Let us state that at least one of them is true: $Z_1^i \vee Z_2^i \vee Z_3^i \vee Z_4^i$.
- Let us state that two on the same column cannot be true: $\neg Z_1^i \vee \neg Z_2^i, \neg Z_1^i \vee \neg Z_3^i, \neg Z_1^i \vee \neg Z_4^i, \ldots, \neg Z_3^i \vee \neg Z_4^i$.
- Horizontal attack. For $i \neq j$: $\neg Z_1^i \vee \neg Z_1^j, \neg Z_2^i \vee \neg Z_2^j, \neg Z_3^i \vee \neg Z_3^j, \neg Z_4^i \vee \neg Z_4^j$.
- Diagonal attack. For $i \neq j$. Let $i = 2, j = 4$, then: $\neg Z_1^2 \vee \neg Z_3^4, \neg Z_2^2 \vee \neg Z_4^4, \neg Z_3^2 \vee \neg Z_1^4, \neg Z_4^2 \vee \neg Z_2^4$.

SAT Reduction

4 Queens

- We should state that the variables X_1, X_2, X_3, X_4 have values in $\{1,\ldots,4\}$.
- For each of them (let us focus on the generic X_i) we introduce 4 Boolean Variables: $Z_1^i, Z_2^i, Z_3^i, Z_4^i$.
- Let us state that at least one of them is true: $Z_1^i \vee Z_2^i \vee Z_3^i \vee Z_4^i$.
- Let us state that two on the same column cannot be true: $\neg Z_1^i \vee \neg Z_2^i, \neg Z_1^i \vee \neg Z_3^i, \neg Z_1^i \vee \neg Z_4^i, \ldots, \neg Z_3^i \vee \neg Z_4^i$.
- Horizontal attack. For $i \neq j$: $\neg Z_1^i \vee \neg Z_1^j, \neg Z_2^i \vee \neg Z_2^j, \neg Z_3^i \vee \neg Z_3^j, \neg Z_4^i \vee \neg Z_4^j$.
- Diagonal attack. For $i \neq j$. Let $i = 2, j = 4$, then: $\neg Z_1^2 \vee \neg Z_3^4, \neg Z_2^2 \vee \neg Z_4^4, \neg Z_3^2 \vee \neg Z_1^4, \neg Z_4^2 \vee \neg Z_2^4$.

SAT

DIMACS FORMAT

- Instances must be written in a ASCII file according to a format (use the suffix .cnf)
- Variables are represented by integer numbers (0 excluded)
- Comments starts with “c”
- There is an initial line starting with “p” storing the number of variables and of clauses.

```
c ****************************************
c SAT Encoding of my favourite problem
c ****************************************
p cnf 256 1200
54 0
-12 85 0
33 161 -200 0
...
```

There are 256 variables, 1200 clauses. The first clauses are $(X_{54}), (\neg X_{12} \vee X_{85}), (X_{33} \vee X_{161} \vee \neg X_{200})$.

SAT Reduction

Exercise: *n* queens

- Write a program (choose your favourite language) that given *n* write the SAT encoding of *n* queens and run a SAT solver on it.
- (If you want) write a post-processor for printing the solution in a readable way.
- You can use any SAT solver (eg Minisat `http://minisat.se/`)
- We test `clingo` (an Answer Set Programming solver) that we can use as SAT solver
- Download clingo at `http://potassco.sourceforge.net/`
- Launch `clingo --mode=clasp filename.cnf`

SAT Reduction

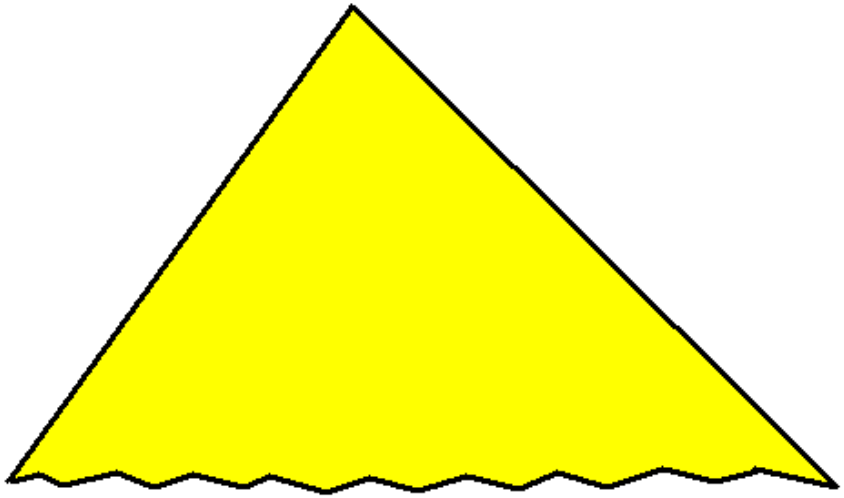
Exercise: SUDOKU

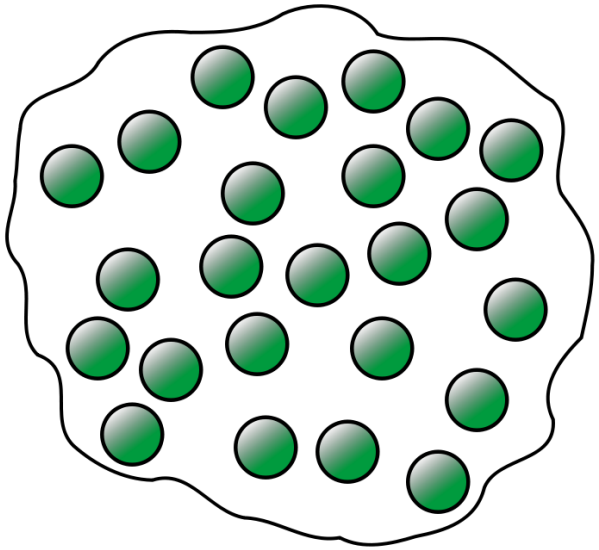
Homework: encode Sudoku problem and write a c program that generates the corresponding CNF formula

- Encoding numbers is tricky (we use boolean variables!)
- Any change to the model would force us to rewrite the (C/JAVA/Python/etc) program that generates the file (which is typically difficult to be read)
- Later on we will see how SAT solvers work

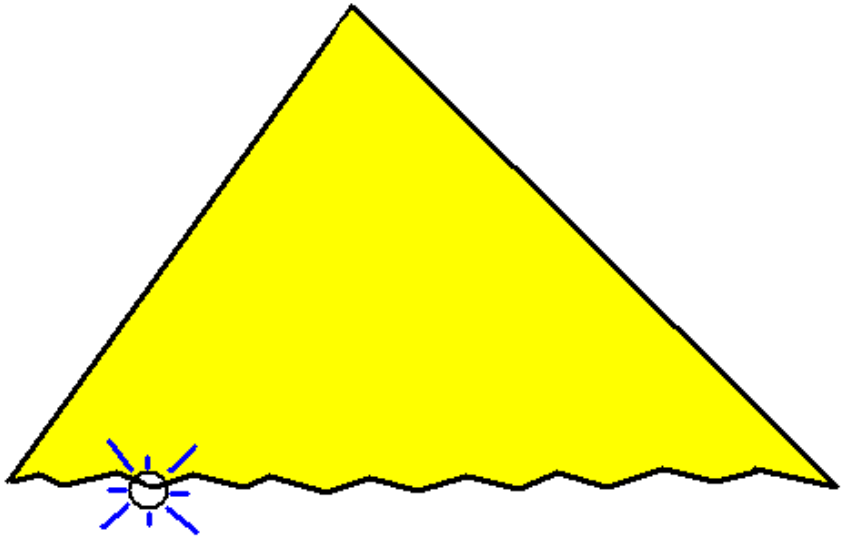
Local Search

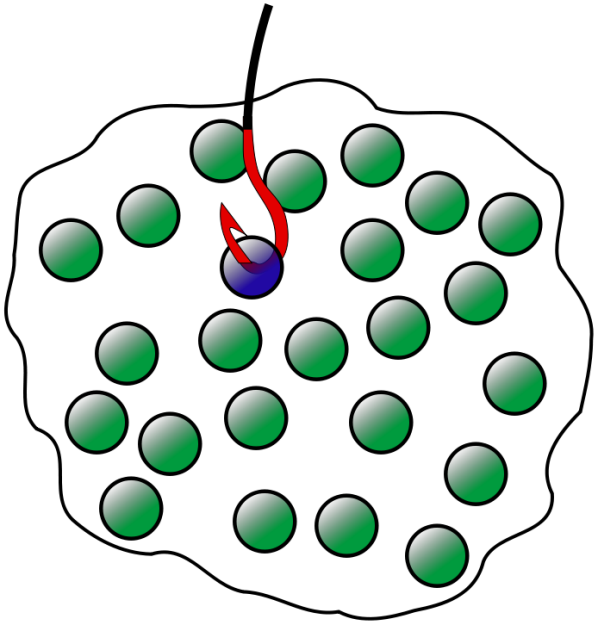
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . .).
All of them are based on random choices.



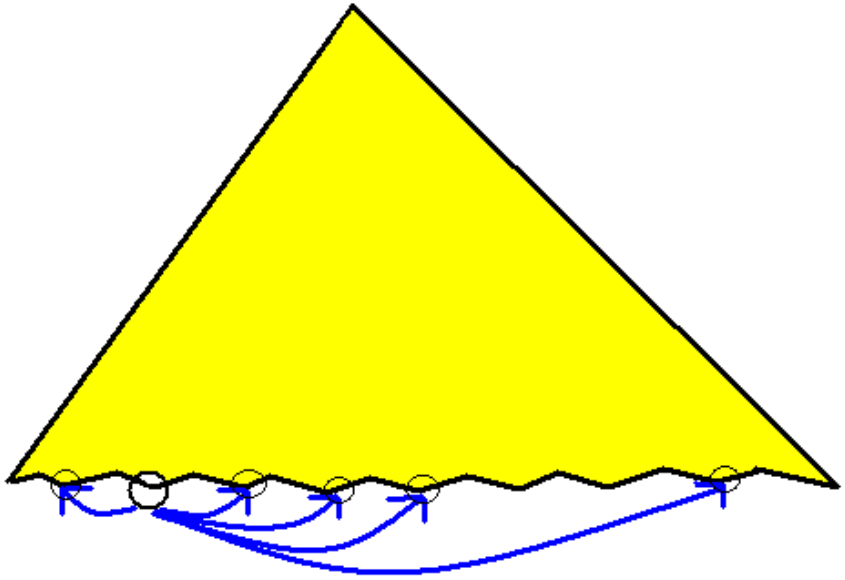


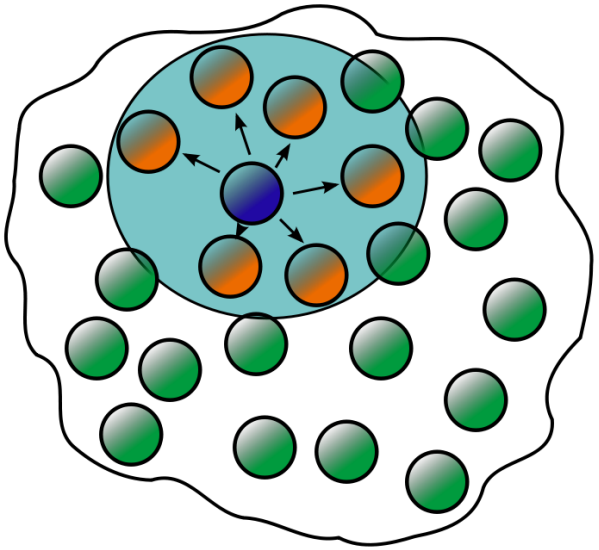
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . .).
All of them are based on random choices.





Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . .).
All of them are based on random choices.

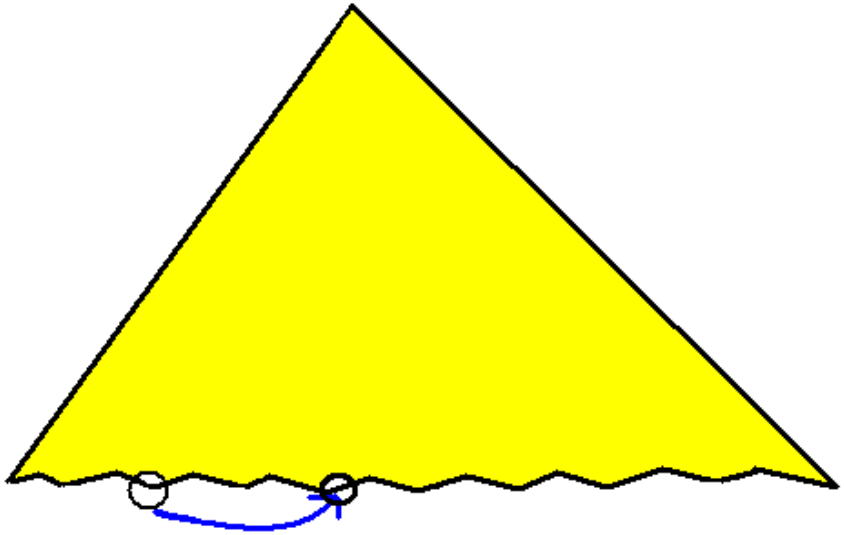


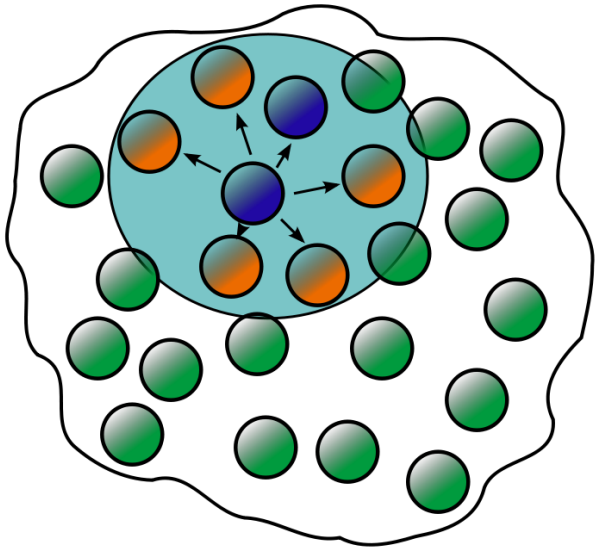


Local Search

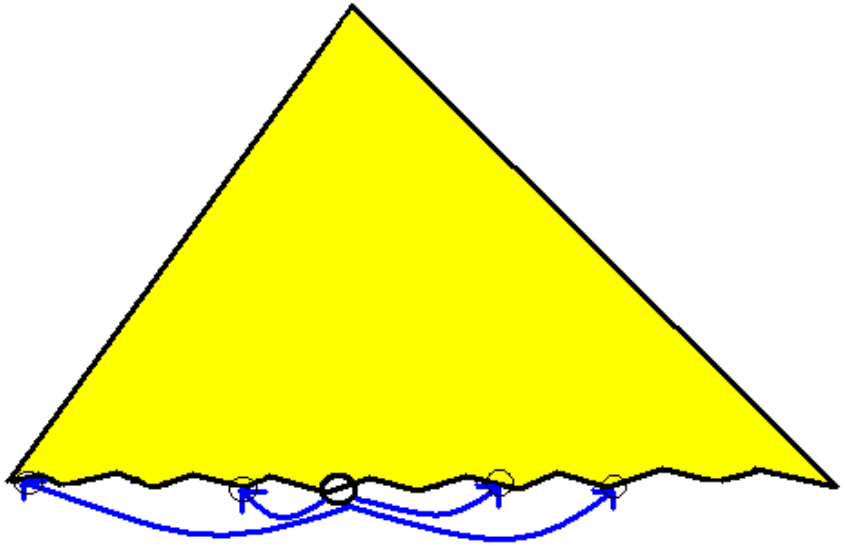
Solving COPs

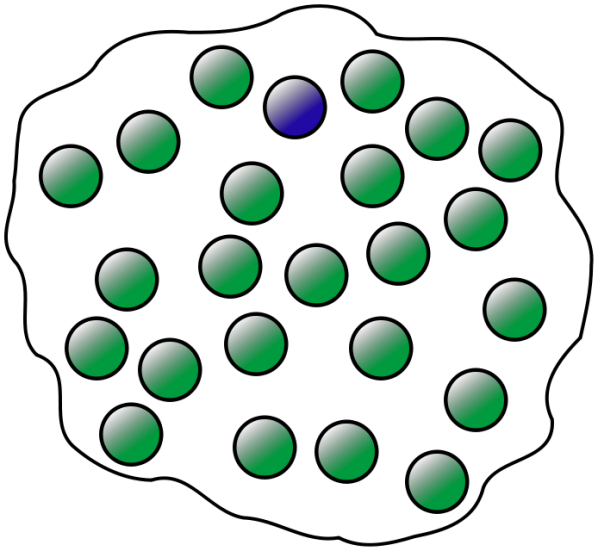
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . .).
All of them are based on random choices.



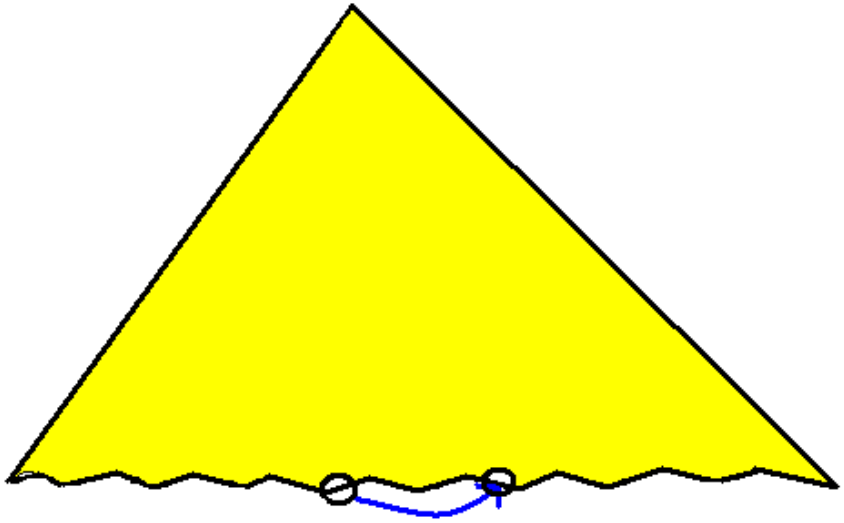


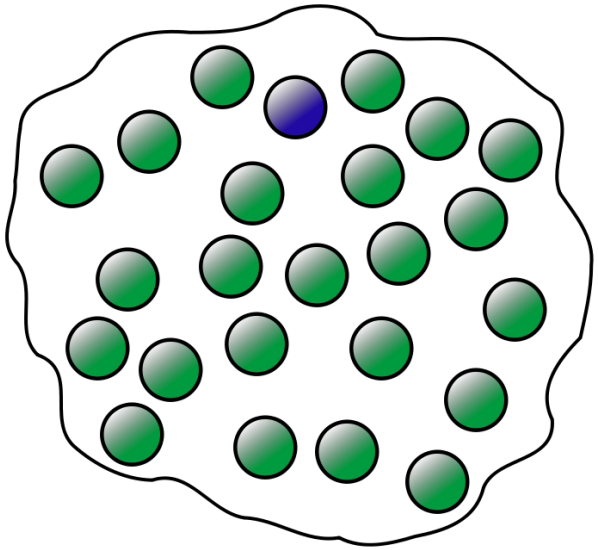
Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . .).
All of them are based on random choices.



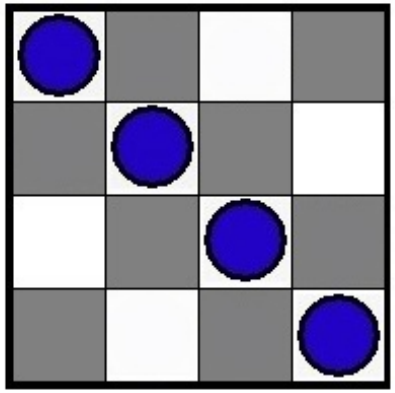


Given a possible solution, the algorithm performs some changes, i.e. evaluate other leaves.
There are several techniques (Montecarlo, Simulated Annealing, Particle Swarm, Ant Colony, Genetic, . . .).
All of them are based on random choices.



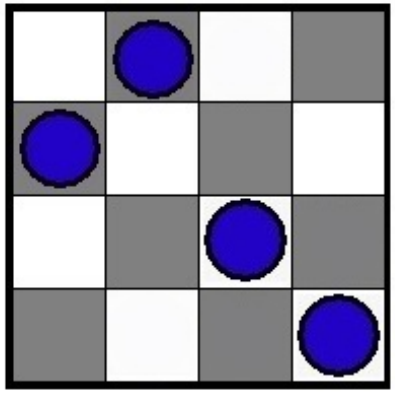


- LS methods can be applied to CSP.
- Remove some constraints and count a penalty anytime those constraints are violated. We find more admissible solutions!
- Less constraints $\rightarrow$ easier to find an initial solution
- Then run the LS algorithm. If the optimum has value 0 then all the constraints are satisfied.
- Let us see an example with the Queens. Let us remove the diagonal constraints and define as f the number of attacks (i.e. number of violated diagonal constraints).

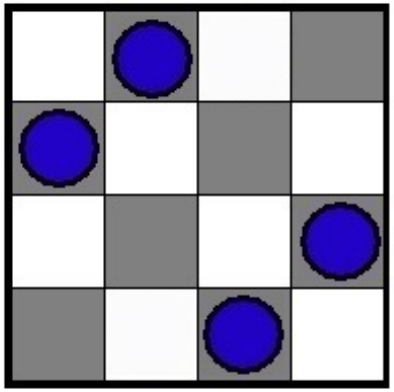


12 attacks

$\Rightarrow$ swap(1,2)

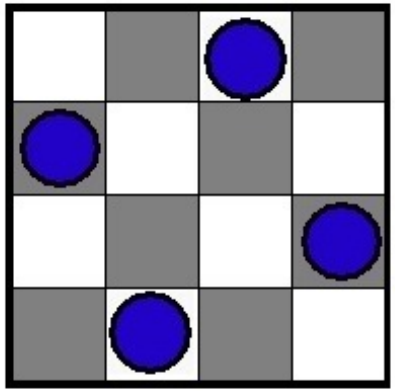


4 attacks $\Downarrow$ swap(3,4)

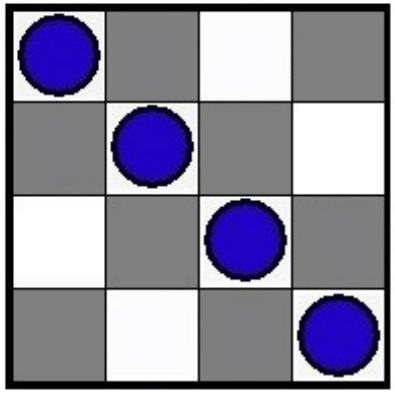


8 attacks

$\Leftarrow$ swap(2,3)

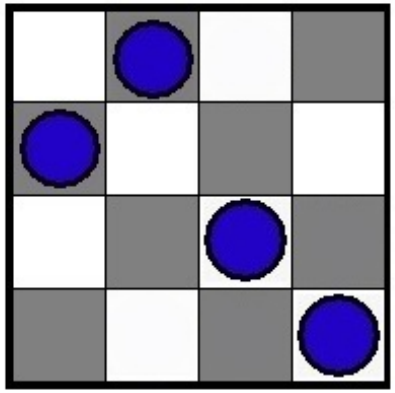


0 attacks

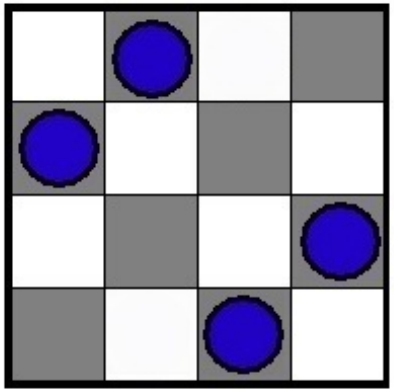


12 attacks

$\Rightarrow$ swap(1,2)

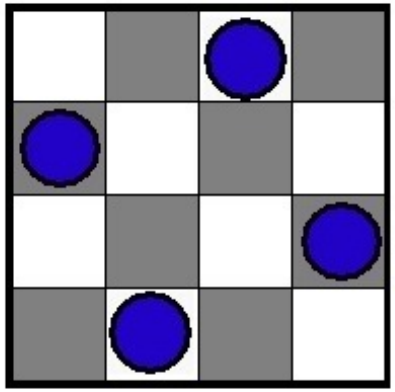


4 attacks $\Downarrow$ swap(3,4)

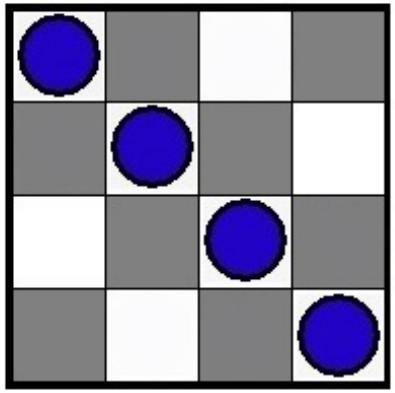


8 attacks

$\Leftarrow$ swap(2,3)

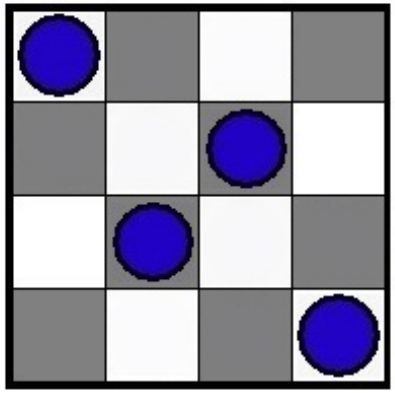


0 attacks

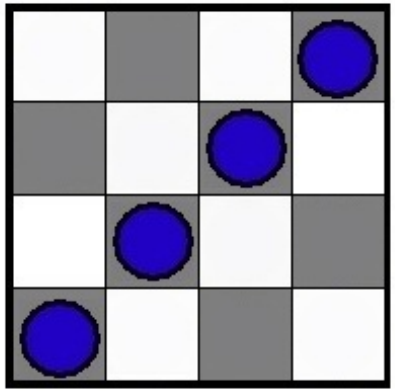


12 attacks

$\Rightarrow$ swap(2,3)

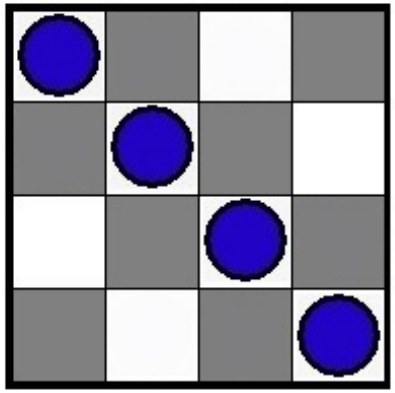


4 attacks $\Downarrow$ swap(1,4)



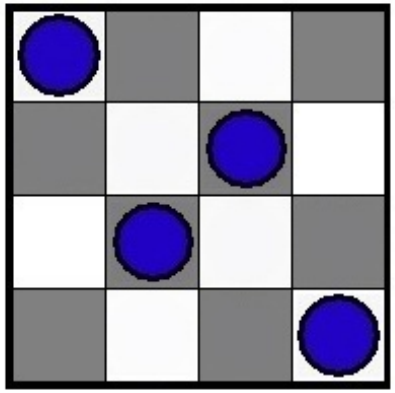
12 attacks

Not every choice improves the violations (+ local minima!)

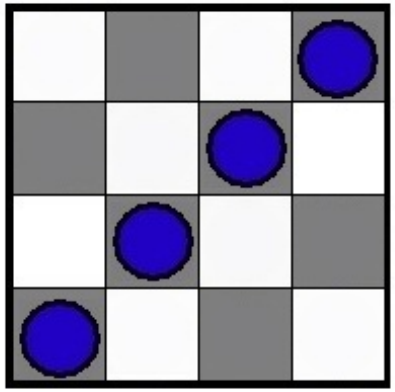


12 attacks

$\Rightarrow$ swap(2,3)



4 attacks $\Downarrow$ swap(1,4)



12 attacks

Not every choice improves the violations (+ local minima!)

Define:

- An algorithm for finding the initial solution
- A function N that computes the Neighborhood of a solution σ that identifies a set $N(\sigma)$ of feasible solutions similar to σ
- If possible, the Neighborhood is characterized by a simple *move* (e.g., change/swap of values between two variables)
- Moreover:
  - How to select a move
  - How to explore the Neighborhood
  - How to avoid undesirable solutions/situations (e.g., loops)
  - How/When halting the computation (typically, after a fixed number of non-improving steps)

- Given a solution σ, compute $N(\sigma)$.
  - Hill Climbing: choose $\sigma' \in N(\sigma)$ minimizing locally f, namely $f(\sigma') = \min\{f(\mu) \,:\, \mu \in N(\sigma)\}$
  - Simulated Annealing:
    - Let T (temperature) be a gloabal variable slowly decreasing during computation.
    - $*$ Chose randomly $\sigma' \in N(\sigma)$.
    - If $f(\sigma') < f(\sigma)$ take σ' as new solution.
    - Otherwise, generate a random number $p \in [0, 1]$.
    - Let F be a function in $[0, 1]$ that increases as T decreases and that also computes the distance between $f(\sigma)$ and $f(\sigma')$.
    - If $p > F(T, f(\sigma), f(\sigma'))$ then take σ' as new solution, otherwise go to $*$ (choose a new random $\sigma' \in N(\sigma)$ and repeat last items)

- Monte Carlo: It is a sort of Simulated Annealing with fixed temperature and $F = 0.5$.
  The name Monte Carlo was coined in the 1940s by Stanislaw Ulam, Enrico Fermi, John von Neumann, and Nicholas Metropolis among others working on nuclear weapon projects in the Los Alamos Labs.
- Tabu Search: It is a Simulated Annealing, with the difference that you store a list (tabu list) if the last k solutions in order to avoid loops.
- Others ...

- Monte Carlo: It is a sort of Simulated Annealing with fixed temperature and $F = 0.5$.
  The name Monte Carlo was coined in the 1940s by Stanislaw Ulam, Enrico Fermi, John von Neumann, and Nicholas Metropolis among others working on nuclear weapon projects in the Los Alamos Labs.
- Tabu Search: It is a Simulated Annealing, with the difference that you store a list (tabu list) if the last k solutions in order to avoid loops.
- Others ...

Local Search

When?

- Discrete optimization problems where the notion of swap is natural (e.g., timetables, scheduling, etc.)
- Continuous problems (e.g., simulations)
- Easy to implement. However, better using an ad hoc tool (e.g. EasyLocal) and focusing on the problem and on the meta-heuristics.